

Entwicklerdokumentation

1 Hardware

1.1 Schaltplan mit Sicherheitseinrichtungen

Die Ladesäule ist folgendermaßen (Abb. 1) mit Leitungsschutzschaltern gegen Überlast und RCDs für den Schutz des Menschen abgesichert. Dieser ist im Doku-Ordner gesondert als PDF-Datei zu finden.

Ladesäule R-Bau L3E

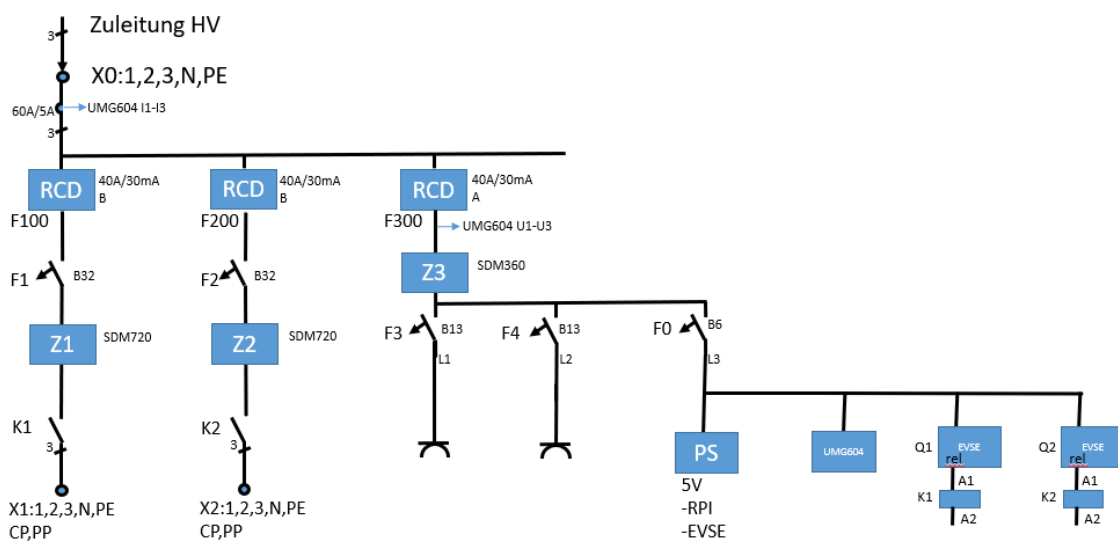


Abbildung 1: Sicherheitseinrichtungen in der Ladesäule

1.2 Verdrahtungsplan Kommunikation

Folgende Abbildung 2 zeigt die Verdrahtung für die Kommunikation des Raspberry mit den Laderegler, den Messgeräten und dem Bildschirm.

Der Bildschirm bezieht seine Spannungsversorgung über die GPIOs des Raspberry. Außerdem sind die beiden mit einer Flachbandleitung für die Datenübertragung miteinander verbunden. Dieser Verdrahtungsplan ist ebenfalls als gesondertes Dokument in dem Doku-Ordner zu finden.

Der in der EVSE verbaute PP-Widerstand hat 220 Ohm, was der EVSE einen Hardware-begrenzten maximalen Strom von 32A signalisiert.

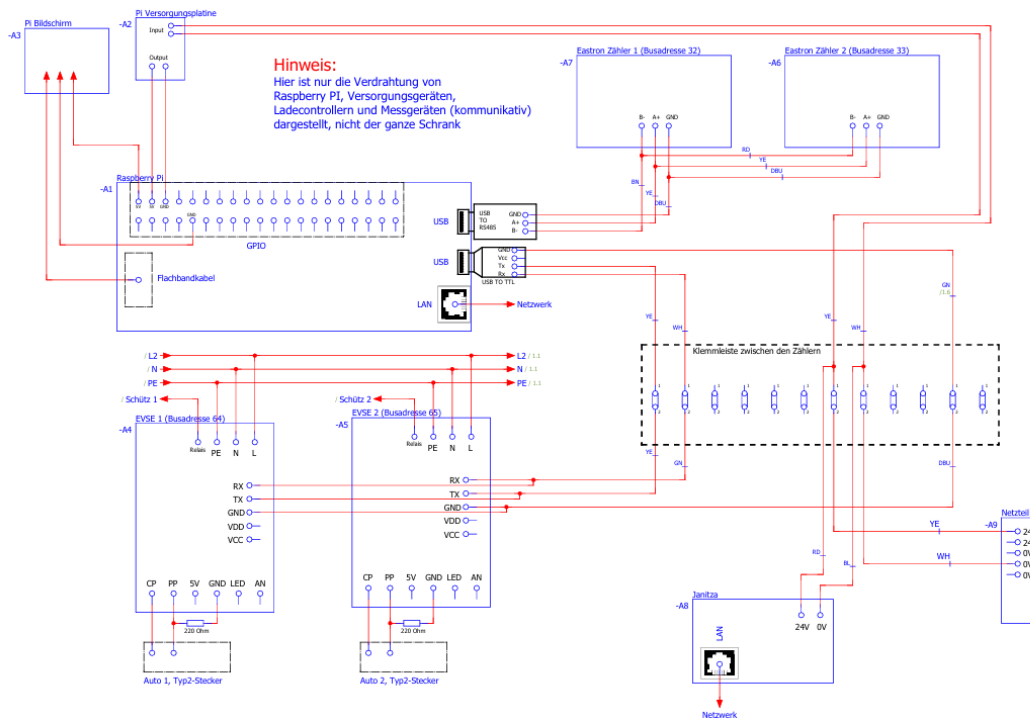


Abbildung 2: Verdrahtungsplan der Kommunikation zwischen den Geräten

Das Janitza (Netzanalysegerät) kommuniziert direkt über Modbus-TCP und braucht deswegen keine weitere Konfiguration.

Die EVSE-Module müssen jedoch erst in den Modbus-Modus versetzt werden und diesen muss dann auch noch eine Modbus-Adresse gegeben werden. Das geht am einfachsten persistent über die Software EVSE-Control. Eine Anleitung für dieses Tool ist im Dokumentationsordner zu finden genauso wie das Datenblatt des Controllers selbst.

Die Aktivierung der Modbus-Funktionalität lässt sich folgendermaßen durchführen und überprüfen:

1. Verdrahtung: Adapterkabel wie in der EVSE-Control-Anleitung beschrieben an den Controller anschließen (**USB-seitig noch nicht anstecken!**), jeweils eine Leitung (beidseitig entsprechend abisoliert) an Pin „GND“ und „AN“ anklemmen und netzseitig L, N und PE anklemmen
2. EVSE-Control öffnen („Error: Keine Verbindung!“ zu sehen im Statusfeld)
3. Netzspannung am Controller abschalten und wieder einschalten und danach innerhalb der ersten 3 Sekunden mindestens 5-mal „GND“ und „AN“ kurzschließen (besser öfters!)
4. USB-to-Serial Adapter am Rechner einstecken (vermutlich noch keine Verbindung)
5. EVSE-Control schließen und erneut öffnen
6. „OK“ sollte im Statusfeld zu sehen sein

Falls es nicht sofort funktioniert, ist das nicht direkt problematisch. Der Prozess muss dann schlichtweg wiederholt werden.

Sobald der Controller im Modbus-Modus ist, kann in das Modbus-Register 2001 die Modbus-Adresse geschrieben werden. Achtung: Wenn dort der Wert 0 geschrieben wird, schaltet das

EVSE-Modul die Modbus-Kommunikation wieder persistent ab und man muss den ganzen Prozess zum Modbus-Aktivieren erneut durchlaufen.

In der Ladesäule haben die beiden Module die Modbus-Adressen 64 und 65.

Für die beiden Eastron-Stromzähler können die Kommunikationsparameter manuell am Display konfiguriert werden (siehe Datenblatt im Dokumentationsordner). Die Zähler haben die Bus-Adressen 33 und 34.

Wichtig zu erwähnen ist noch, dass zwar beide Geräte (Stromzähler und Controller) per Modbus-RTU Protokoll mit dem Raspberry Pi kommunizieren, sich jedoch in im Hinblick auf die physikalische Verbindung unterscheiden, weshalb auch zwei verschiedene Adapter verwendet werden und zwei Bussysteme. Die Controller verwenden TTL-basierte Übertragung wohingegen die Zähler per RS485 (Differenzsignal) angebunden sind.

Sollte es in irgendeiner Form Kommunikationsprobleme aufgrund von Modbus geben, sind Tools wie zum Beispiel ModbusPoll für Windows empfehlenswert.

1.3 Kartenleser

Es wurde der Kartenleser TWN4 Multitech eingebaut, mit dem die HM-Karten von Studierenden und Mitarbeitern eingelesen werden können. Zu dem Kartenleser gibt es einen DevPack Ordner sowie Video-Tutorials des Herstellers, der sich auf der Webseite des Herstellers ELATEC herunterladen lässt, wenn man sich für den Newsletter anmeldet:

<https://www.elatec-rfid.com/de-de/elatec-software#/>

Der Ordner befindet sich aber auch in der bereitgestellten Doku.

Auf dem Kartenleser lässt sich unterschiedliche Firmware aufspielen, wodurch der Kartenleser seine Schnittstelle verändern kann. Die Firmware wird von Firma ELATEC bereitgestellt, könnte aber auch von Grund auf selbst programmiert werden. Im Ordner *TWN4DevPack492->Firmware* finden sich unterschiedliche Firmware-Dateien. Derzeit ist auf dem Kartenleser die *TWN4_xCx492_STD205_MULTI_CDC_Standard.bix* Firmware aufgespielt. Somit hat der Kartenleser eine CDC-Schnittstelle. Es gäbe auch die Möglichkeit einer HID-Schnittstelle, jedoch scheint Node-Red damit Probleme zu haben.

Umprogrammieren lässt sich der Kartenleser mit dem *AppBlaster.exe* Tool, welches sich ebenfalls im Ordner befindet.

1.4 Raspberry Pi

Auf dem Raspberry Pi wurde zunächst ein Raspberry PI OS Lite sowie folgende Applikationen installiert. Installationsanleitungen finden sich auf den jeweiligen Seiten der Applikationen:

- PostgreSQL (Datenbank zum Speichern der Benutzer sowie Werte)
- Node-Red (für die Datenverarbeitung)
- nano (nano)
- X Window Systems (Display Manager)

Es befindet sich eine SD-Karte im Labor, auf dem die Programme installiert sind und mit dem ein Raspberry Pi booten sollte. Zudem hat die SD-Karte eine statische IP-Adresse für das Ethernet-

Netzwerk eth0 von 192.168.137.2, einer Netzmaske von 255.255.255.0 und einem DNS-Server von 8.8.8.8. Damit kann man auch von seinem Laptop auf den Raspberry Pi zugreifen, wenn man ihm eine IP-Adresse von 192.168.137.x sowie die gleichen Konfigurationen vergibt.

Wie man auf dem Raspberry in der Ladesäule zugreift, ist unter Gliederungspunkt 3 erläutert.

Login-Daten (auch per ssh):

Benutzer: admin

Passwort: lamaadmin1234

2 Node-Red Programmierung (Software)

In Node-Red findet die Kommunikation mit den Messgeräten, den Laderegler sowie dem Kartenleser statt. Der Flow aus dem Wintersemester 24/25 wurde um die Authentifizierung und dem Messen der einzelnen Phasenwerte der Typ 2-Anschlüsse erweitert. Es müssen folgende Nodes installiert werden, damit der Kommunikationsflow funktioniert:

- node-red-contrib-buffer-parser
- node-red-contrib-modbus
- node-red-contrib-postgresql
- node-red-dashboard
- node-red-node-email
- node-red-node-serialport
- node-red-node-smooth

Node-Red lässt sich ebenfalls direkt auf Windows installieren, so dass man seinen Flow auch lokal testen kann, bevor man ihn auf einen Raspberry Pi aufspielt.

Damit die Kommunikation per USB_Serial richtig funktioniert, muss noch die Umgebungsvariable UV_USE_IO_URING=0 beim Betrieb von Node-Red gesetzt sein. Am einfachsten geht das, in dem eine Datei ~/.node-red/environment erstellt wird, in dem UV_USE_IO_URING=0 steht.

Dieses File wird dann automatisch beim Start von Node-Red beachtet und der korrekte Wert von UV_USE_IO_URING gesetzt.

Autologin beim Boot

Damit der Autologin und die Anzeige des Desktops funktioniert, muss das noch in den Systemeinstellungen angepasst werden. Dafür muss per Terminal der Befehl sudo raspi-config ausgeführt werden.

Dann per Pfeiltasten [up], [down] und [enter] navigieren:

1 System Options -> S5 Boot / Auto Login -> B4 Desktop Autologin

Die Auswahl mit einem Enter bestätigen, dann auf der Startseite per [Tab] zu Finish navigieren, mit [enter] bestätigen und ggf. neustarten.

Herunterladen für Windows

Um Flows zu testen empfiehlt es sich, dass nicht auf dem Raspberry, sondern auf seinem eigenen Windows System zu machen.

Eine Möglichkeit wäre node-red in einem Container laufen zu lassen (z.B. Docker Desktop). Das ist allerdings nur zu empfehlen, wenn man sich in der Thematik auskennt oder einlesen will. Unkomplizierter ist dieser Weg ohne Docker:

Node.js installieren

- Geh auf <https://nodejs.org> und lade die **LTS-Version** herunter
- Installiere sie mit den Standard-Einstellungen
- Prüfe die Installation in der Eingabeaufforderung (cmd): `node -v`

→ Du solltest eine Versionsnummer sehen, z. B. v20.11.1.

Node-RED installieren

- Öffne die Eingabeaufforderung (cmd)
- Gib ein: `npm install -g --unsafe-perm node-red`
- Warte, bis die Installation durchgelaufen ist

Node-RED starten

- Starte Node-RED mit: `node-red`
- Am Ende steht eine Zeile wie:

Server now running at http://127.0.0.1:1880/

Node-RED im Browser öffnen

- Gib im Browser deiner Wahl ein: `http://localhost:1880`

→ Jetzt bist du in der Node-RED-Oberfläche.

Importiere nun einen Flow deiner Wahl oder erstelle einen eigenen.

Normalerweise ist keine zusätzliche Einstellung nötig. Es sollten bei einem importierten Flow darauf geachtet werden alle nötigen nodes in der Palette heruntergeladen wurden.

Zum Node-RED beenden

- Drück in der Eingabeaufforderung **Strg + C**

Herunterladen für Raspberry Pi/ Debian Linux

Falls ein neuer Raspberry Pi eingerichtet werden soll oder für Linux-Rechner

System aktualisieren

- Öffne ein Terminal und gib ein: `sudo apt update && sudo apt upgrade -y`

Node-RED installieren

Die Entwickler von Node-RED bieten ein bequemes Installationsskript an. Tippe:

```
bash <(curl -sL https://raw.githubusercontent.com/node-red/linux-installers/master/deb/update-nodejs-and-nodered)
```

- Das Script installiert Node.js und Node-RED und richtet Node-RED als Dienst ein, der beim Booten startet.

Node-RED starten

Nach der Installation kannst du Node-RED sofort starten: *node-red-start*

Wenn Node-RED als Dienst laufen soll, damit es automatisch startet:

```
sudo systemctl enable nodered.service
```

```
sudo systemctl start nodered.service
```

Node-RED im Browser öffnen

Öffne deinen Browser auf einem anderen Gerät im gleichen Netzwerk. Gib ein:

```
http://<IP-Adresse-deines-Pi>:1880
```

→ Jetzt siehst du die Node-RED-Oberfläche.

Node-RED stoppen

Im Terminal kannst du Node-RED stoppen mit: *node-red-stop*

2.1 Optimierungs-Algorithmus

Der Code der Optimierung:

```
const max_amps_per_phase = 33;
```

```
// Fahrzeugzustände lesen
```

```
let state_1 = flow.get("vehicle_1_state");
```

```
let state_2 = flow.get("vehicle_2_state");
```

```
let auto_1_active = (state_1 > 1 && state_1 < 5 && (flow.get("auth_1") || flow.get("auth_off")));
```

```
let auto_2_active = (state_2 > 1 && state_2 < 5 && (flow.get("auth_2") || flow.get("auth_off")));
```

```
let active_autos = (auto_1_active ? 1 : 0) + (auto_2_active ? 1 : 0);
```

```
// Schuko-Strom berechnen
```

```
let schuko_L1 = flow.get("I_L1") - (flow.get("SDM1_IL1") + flow.get("SDM2_IL1"));
```

```
let schuko_L2 = flow.get("I_L2") - (flow.get("SDM1_IL2") + flow.get("SDM2_IL2"));
```

```
let schuko_L3 = flow.get("I_L3") - (flow.get("SDM1_IL3") + flow.get("SDM2_IL3"));
```

```
// Reserven pro Phase
```

```

let reserve_L1 = max_amps_per_phase - schuko_L1;
let reserve_L2 = max_amps_per_phase - schuko_L2;
let reserve_L3 = max_amps_per_phase - schuko_L3;

let gemeinsame_reserve = Math.floor(Math.min(reserve_L1, reserve_L2, reserve_L3));

// Grundverteilung
let limit_1 = 0;
let limit_2 = 0;

if (active_autos === 2) {
    limit_1 = Math.floor(gemeinsame_reserve / 2);
    limit_2 = Math.floor(gemeinsame_reserve - limit_1);
} else if (auto_1_active) {
    limit_1 = gemeinsame_reserve;
} else if (auto_2_active) {
    limit_2 = gemeinsame_reserve;
}

// Stromverbrauch der am stärksten belasteten Phase lesen
let sdm1_Lx = Math.max(flow.get("SDM1_IL1"), flow.get("SDM1_IL2"), flow.get("SDM1_IL3"));
let sdm2_Lx = Math.max(flow.get("SDM2_IL1"), flow.get("SDM2_IL2"), flow.get("SDM2_IL3"));

// Dynamische Umverteilung (beide Richtungen)
let ungenutzt_1 = limit_1 - sdm1_Lx;
let ungenutzt_2 = limit_2 - sdm2_Lx;

// Dynamische Umverteilung aktiviert für den Fall dass beide Autos aktiv laden
if ((flow.get("vehicle_1_state") + flow.get("vehicle_2_state")) === 6) {
    if (ungenutzt_1 > 1 && sdm1_Lx > 3) {
        let abgabe = Math.floor(ungenutzt_1);
        limit_1 -= abgabe;
    }
}

```

```

    limit_2 += abgabe;
} else if (ungenutzt_2 > 1 && sdm2_Lx > 3) {
    let abgabe = Math.floor(ungenutzt_2);
    limit_2 -= abgabe;
    limit_1 += abgabe;
}
}

//falls ein Auto present und das andere lädt
if ((flow.get("vehicle_1_state") + flow.get("vehicle_2_state")) === 5) {
    //falls beide autorisiert (eins lädt und das andere soll auch laden)
    if(active_autos ===2){
        limit_1 = Math.floor(gemeinsame_reserve / 2);
        limit_2 = Math.floor(gemeinsame_reserve - limit_1);
    }

    //falls Auto 1 autorisiert und Auto 2 nicht
    else if (flow.get("auth_1")) {
        limit_1 = Math.floor(gemeinsame_reserve);
    }

    //falls Auto 2 autorisiert und Auto 1 nicht
    else if(flow.get("auth_2")){
        limit_2 = Math.floor(gemeinsame_reserve);
    }
}

// Kleine Reserve nach oben beim kleineren Limit um ggf. eine langsame Stromerhöhung
zuzulassen
if((limit_1 < limit_2) && (limit_2 >= 1)){
    limit_1 += 1;
    limit_2 -= 1;
} else if((limit_1 > limit_2) && (limit_1 >= 1)){
    limit_1 -= 1;
    limit_2 += 1;
}

```

```

}

// Sicherheitsprüfung: keine Phase > 32 A nach Verteilung
function pruefe(limit1, limit2) {
    let total_L1 = schuko_L1 + limit1 + limit2;
    let total_L2 = schuko_L2 + limit1 + limit2;
    let total_L3 = schuko_L3 + limit1 + limit2;
    return (total_L1 <= max_amps_per_phase &&
        total_L2 <= max_amps_per_phase &&
        total_L3 <= max_amps_per_phase);
}

// Fallback falls nach Umverteilung Grenze gerissen würde
while(!pruefe(limit_1, limit_2)) {
    if(limit_1 > 1){
        limit_1 -= limit_1;
    }
    if(limit_2 > 1){
        limit_2 = gemeinsame_reserve - limit_1;
    }
}

// Endgültige Limits setzen
flow.set("evse_1_optimized", auto_1_active ? Math.min(limit_1, 32) : 0);
flow.set("evse_2_optimized", auto_2_active ? Math.min(limit_2, 32) : 0);

return msg;

```

2.2 E-Mail senden

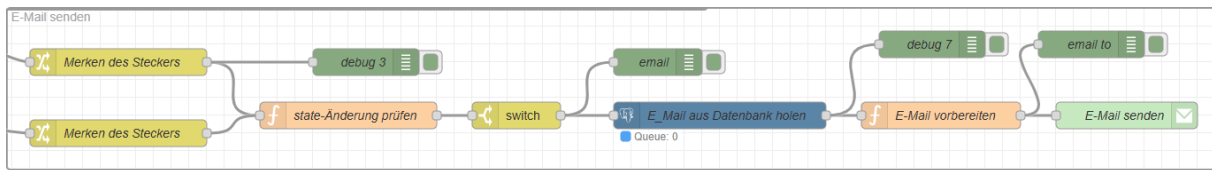


Abbildung 3: node-red Gruppe E-Mail senden

In dem function-node wird überprüft, ob sich der vehicle state von 3 oder 4 (Laden) zu 1 oder 2 geändert (Auto angeschlossen oder EVSE bereit) hat. Dies entspricht dem Fall, dass das Auto wegen einer vollen Batterie das Laden abgeschlossen hat oder der Ladevorgang abgebrochen wurde. Dabei wird ein Unterschied zwischen den beiden Typ 2 Ladesteckern gemacht. Bei den Schuko-Steckern kann dies nicht überprüft werden. Sollte das Laden nun abgeschlossen oder abgerochen sein, so wird die Kartennummer des aktuellen Nutzers und des richtigen Steckers in die msg geschrieben und an den Postgres-node weitergeleitet. Dieser schickt eine Anfrage an die benutzer-Tabelle in der Datenbank und gibt die E-Mail des Nutzers mit der Kartennummer zurück, falls vorhanden. Im folgenden function-node wird die E-Mail vorbereitet und dann mit dem E-Mail-node verschickt.

The screenshot shows the 'Eigenschaften' (Properties) panel for the 'E-Mail senden' node. The fields are as follows:

- Name: E-Mail senden
- An: email@address.com
- Server: smtp.gmail.com
- Port: 465
- ☒ Sichere Verbindung verwenden
- Auth type: Basic
- Benutzer-ID: lamaladesaeule@gmail.com
- Passwort:
- ☒ Überprüfen sie, ob das serverzertifikat gültig ist

Abbildung 4: Einstellung der E-Mail-node

Abbildung 4 zeigt die Einstellung des E-Mail-node.

Die Daten zum Anmelden für den E-mail-node:

Benutzer-ID: lamaladesaeule@gmail.com (entspricht der E-Mail-Adresse)

Passwort: lamaadmin1234 (entspricht auch dem Passwort für die E-Mail-Adresse)
Falls das Passwort nicht funktionieren sollte, wurde ebenfalls ein App-Passwort erstellt. Dieses lautet: fnvh well whjw qijn

In der Vergangenheit ist das Problem aufgetreten, dass das Linux-System den Port 465, über welchen die E-Mail versendet wird, blockiert. Sollte dies auftreten, dann ist die Firewall für diesen Port zu deaktivieren.

2.3 Dashboard

Autostart Bildschirm-Anzeige:

Der Autostart der Anzeige auf dem Bildschirm läuft über ein Linux-AutostartSkript in

~/config/autostart/chrome.desktop

[Desktop Entry]

Type=Application

Exec=bash -c '/usr/bin/sleep 120 && /usr/bin/chromium-browser --

app="http://10.21.6.6:3000/d

2.4 Karte einlesen

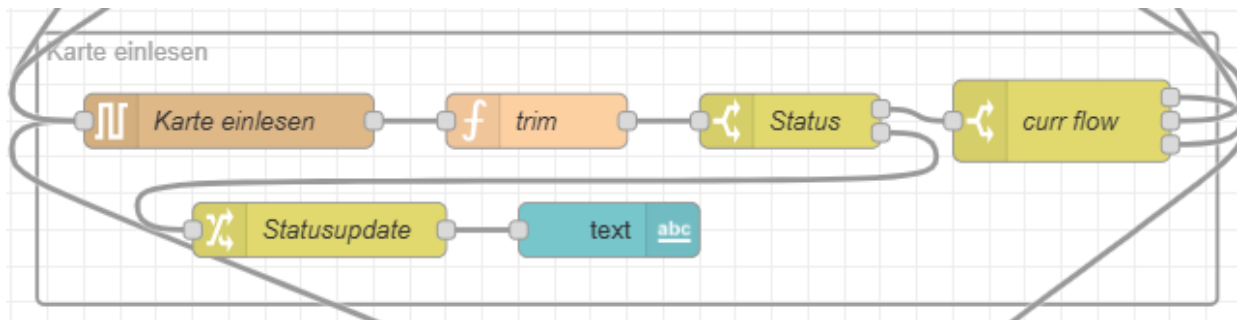


Abbildung 5: node-red Gruppe Karte einlesen

Die Karte wird über den serial-request-node eingelesen. Hierzu wird im vorangegangenen Node ein Request mit 0050020 in den msg.payload an den Kartenleser gesendet. Man hat dann 10s Zeit, um seine Karte einzulesen. Kommt es zu einem Timeout wird dies im status-node überprüft und eine Meldung über das Dashboard ausgegeben.

Da der Kartenleser die Länge der ID manchmal abschneidet, wird im function-node „trim“ die Länge dieser überprüft und angepasst. Dieses Problem ist nur mit den HM-Karten aufgetreten.

Da es zu Problemen gekommen ist, wenn der serial-request-node mehrmals platziert wurde, wurde der Flow so aufgebaut, dass jedes Mal, wenn der Kartenleser angetriggert werden soll, eine Variable curr_flow gesetzt wird, durch den der jeweilige Prozess klar wird. Diese Variable wird im curr flow-node abgefragt, und der Pfad so weitergeleitet.

Derzeit ist die Konvention für curr flow folgende:

- 0: Neuen Benutzer anlegen
- 1: Ladevorgang abbrechen
- 2: Ladevorgang freigeben

2.5 Authentifizierung

Wird ein Auto an einen der EVSE-Stecker angeschlossen, so wird dem Auto über dem EVSE zunächst ein Strom von 0A vorgegeben. Dies erfolgt im Optimierungs-Teil des Flows.

Authentifizieren kann man sich dann über das Display bzw. dem Node-Red-Dashboard. Es wird überprüft, ob der Nutzer in der Datenbank vorhanden ist. Im Freigabe-Node wird zudem die ID des aktuellen Laders in den flow Variablen currloader1 & currloader2 gespeichert.

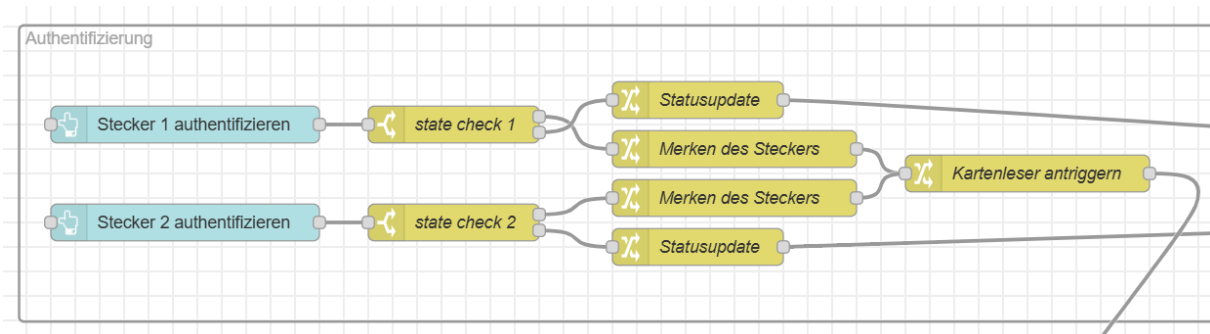


Abbildung 6: node-red Gruppe Authentifizierung 1

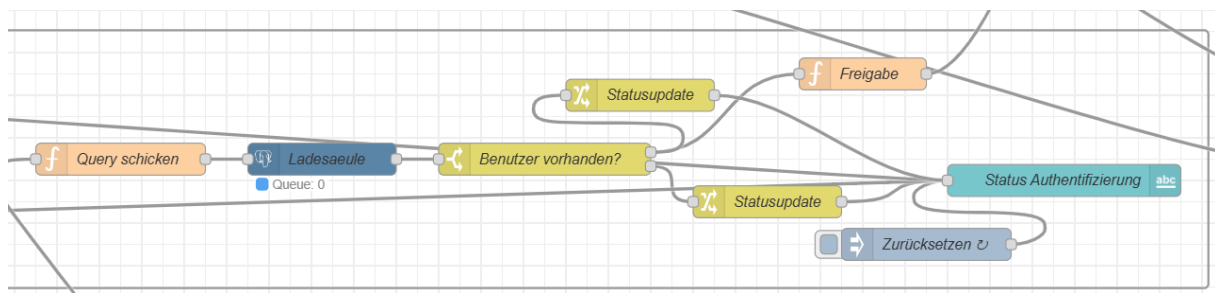


Abbildung 7: node-red Gruppe Authentifizierung 2

Authentifizierung ausschalten

Um die Authentifizierung auszuschalten, wird eine Variable „auth_off“ initialisiert. 1 bedeutet die Authentifizierung ist ausgeschaltet. 0 bedeutet, dass sie angeschaltet ist. Gesetzt wird die Variable über dem inject-Node, indem man den Wert des dortigen msg.payload ändert. Die Authentifizierung wird in der Optimierung, wenn festgesellt wird, ob die Autos aktiv sind oder nicht, abgefragt und „überschreibt“ die Variablen zur Authentifizierung der jeweiligen Variable. Zudem erfolgt im „Ladevorgang freigeben“ direkt die Freigabe und eine Statusnachricht an den Nutzer, dass die Authentifizierung ausgeschaltet ist.

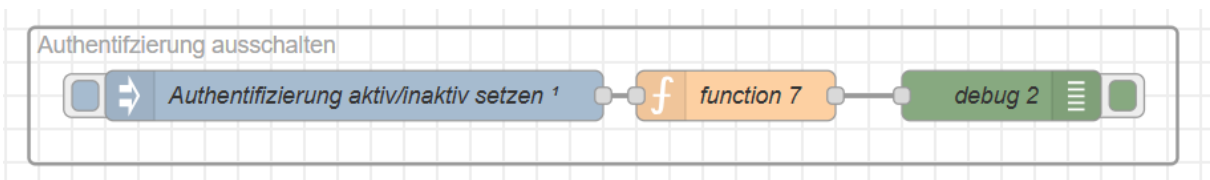


Abbildung 8: Authentifizierungs ausschalten im Flow

```
let auto_1_active = (state_1 > 1 && state_1 < 5 && (flow.get("auth_1") || flow.get("auth_off")));
let auto_2_active = (state_2 > 1 && state_2 < 5 && (flow.get("auth_2") || flow.get("auth_off")));
let active_autos = (auto_1_active ? 1 : 0) + (auto_2_active ? 1 : 0);
```

Abbildung 9: auth_off in der Optimierung

2.6 Ladevorgang abbrechen

Anhand der Flow-Variablen currloader1 & currloader2 wird dann im *Ladevorgang abbrechen* überprüft, ob die eingelesene Karte zu dem gespeicherten Wert und dem gewählten Auto passt. Ist dies der Fall wird dem EVSE ein Strom-Wert von 0A kommuniziert.

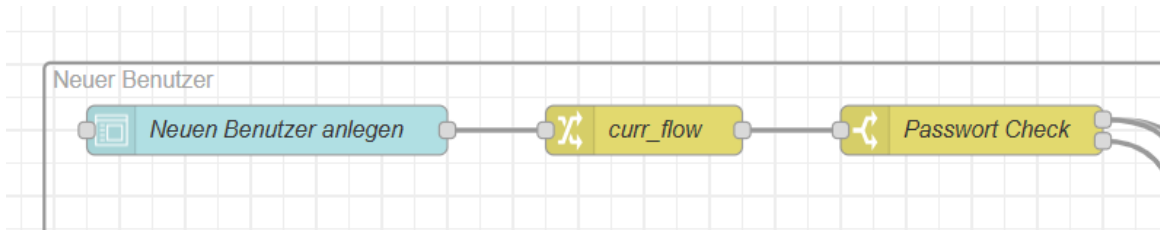


Abbildung 10: node-red Gruppe Ladevorgang abbrechen 1

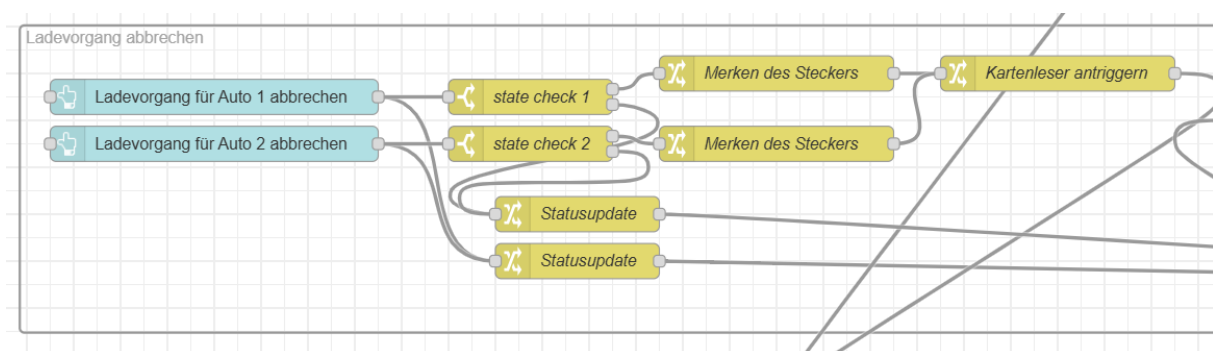


Abbildung 11: node-red Gruppe Ladevorgang abbrechen 2

2.7 Neuer Benutzer

Damit sich ein neuer Benutzer anlegen kann, muss er das Passwort kennen. Derzeit ist es auf 1234 eingestellt. Ändern lässt sich es im Passwort Check-Node, in dem man den zu vergleichen Wert ändert.

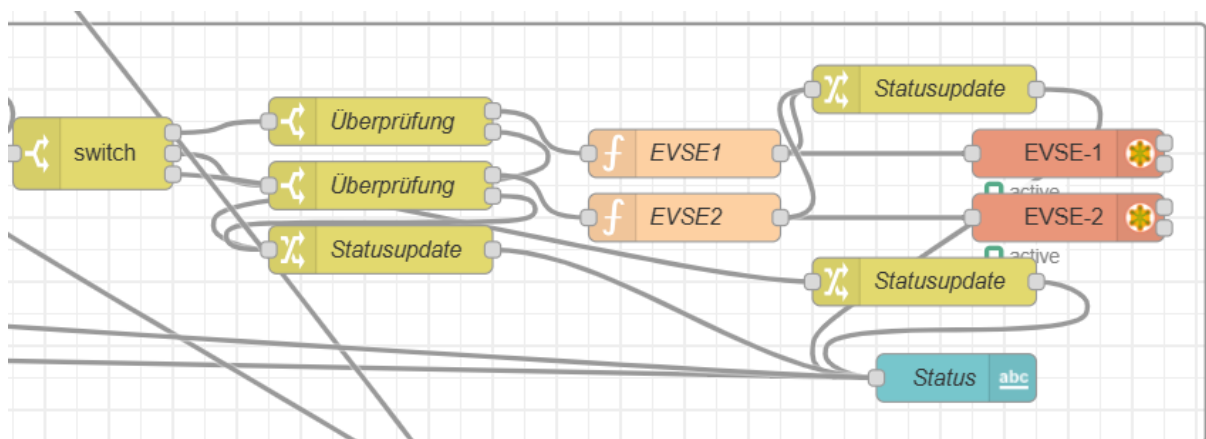


Abbildung 12: node-red Gruppe Neuer Benutzer 1

Name

Eigenschaft

==	a _z 1234	→ 1	x
!=	a _z 1234	→ 2	x

Abbildung 13: Passwort Einstellung

Bei allen Gruppen, die eine Benutzerinteraktion erfordern, werden zudem Statusupdates an das Dashboard kommuniziert.

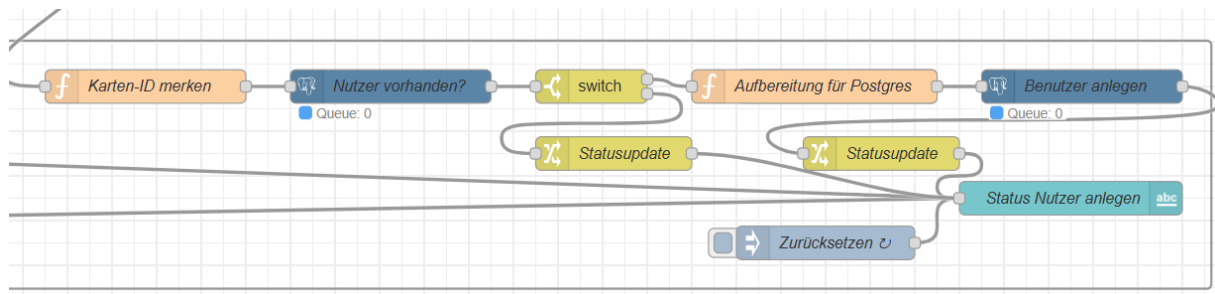


Abbildung 14: node-red Gruppe Neuer Benutzer 2

2.8 Display Tastatur

Für die Display Tastatur wurde ein HTML-Template verwendet und angepasst. Das ursprüngliche Template wurde hier heruntergeladen:

<https://flows.nodered.org/flow/7fb5bc5ae66e6bc1b1c1b8e800bdef51>

Zuletzt gab es noch Probleme, dass die Tastatur während des Tippens an den Anfang der Eingabe zurückspringt.

2.9 Manuelle Steuerung

Die Manuelle Steuerung erlaubt es die Optimierung zu deaktivieren und manuell Ladestromwerte zu setzen. Diese schreibt den Boolean `optimierung_nutzen` in den Flow-State, welcher in der Optimierung abgefragt wird, bevor automatische Werte geschrieben werden.

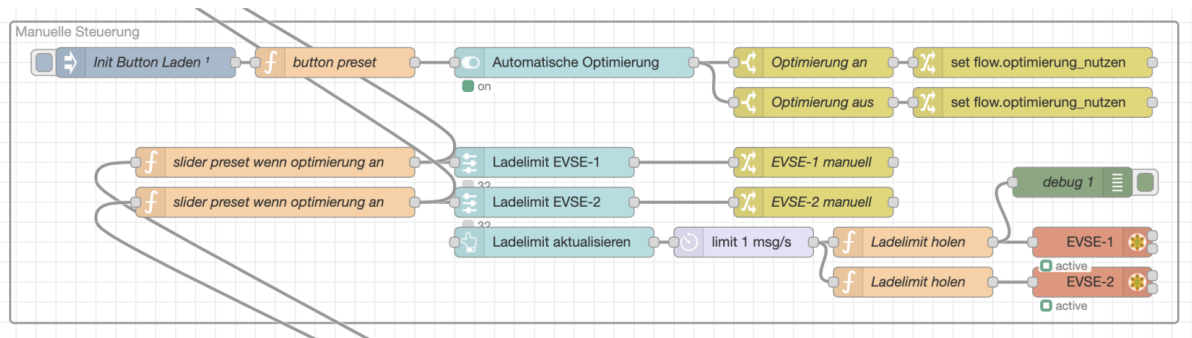


Abbildung 15: node-red Gruppe Manuelle Steuerung

2.10 Admin-Dashboard

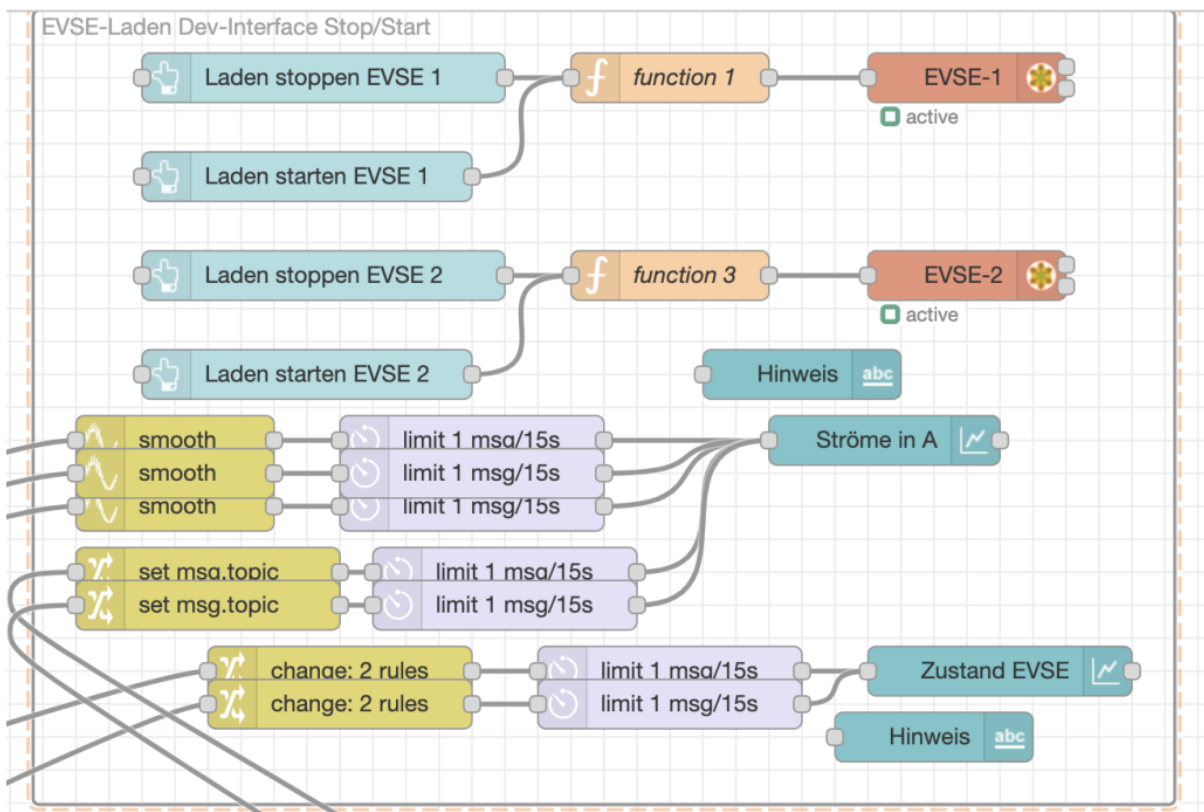


Abbildung 16: node-red Gruppe Admin-Dashboard

Im Admin-Dashboard wird eine längere Historie (~24h) der Stromwerte angezeigt, außerdem gibt es Buttons, um bestimmte Werte in Modbus-Register der EVSE-Module zu schreiben, um zum Beispiel den Ladevorgang manuell zu stoppen.

Das “Admin-Interface” für Enduser ist unter <http://10.21.6.6:1880/ui/#/1> erreichbar.

2.11 Janitza

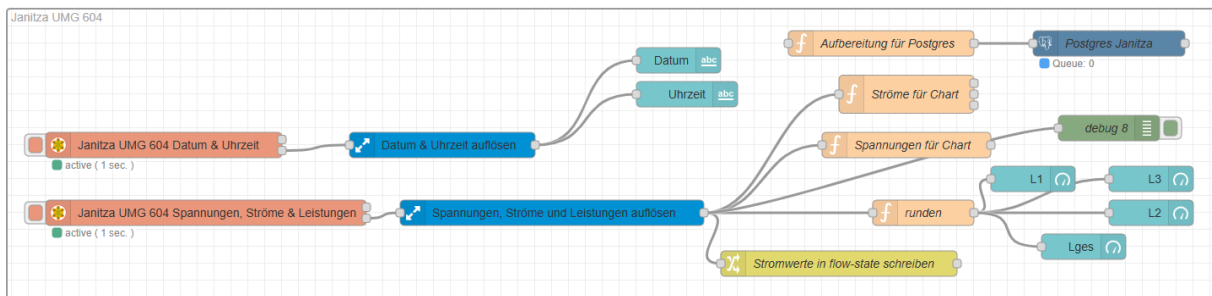


Abbildung 17: node-red Gruppe Janitza (Messgerät)

In dem Janitza-Flow werden die Daten aus der Janitza ausgelesen, dekodiert und angezeigt. Die Strom-Werte wird außerdem direkt in den Flow-State geschrieben, um auch ohne eine direkte “Node-Red-Verbindung”, insbesondere auch asynchron, auf die Daten zugreifen zu können.

2.12 EVSE

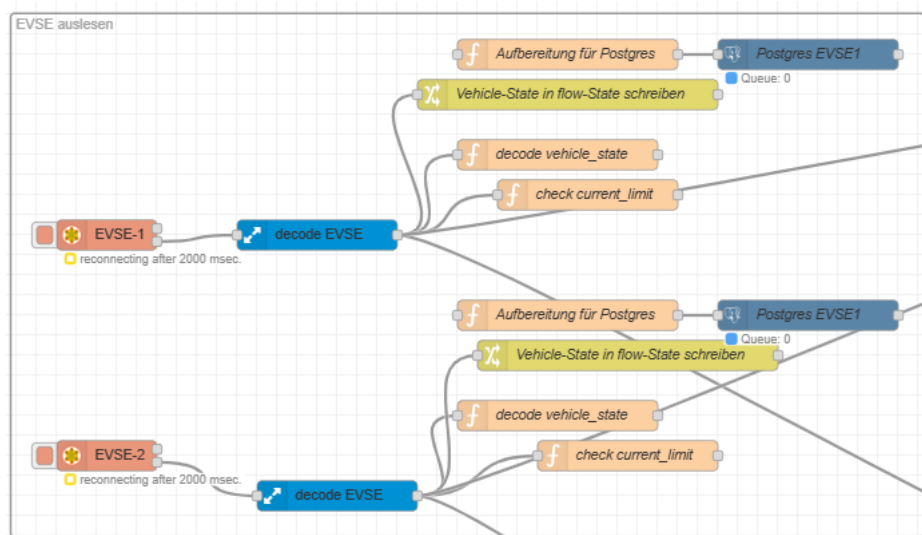


Abbildung 18: node-red Gruppe für die EVSE (Laderegler)

In dem EVSE-Auslesen-Flow werden die Daten aus beiden EVSE-Modulen ausgelesen, dekodiert und angezeigt. Gleichzeitig löst eine Veränderung des Fahrzeugzustandes hier auch die Optimierung aus. Der Vehicle-State wird außerdem direkt in den Flow-State geschrieben, um auch ohne eine direkte “Node-Red-Verbindung”, insbesondere auch asynchron, auf die Daten zugreifen zu können.

2.13 Eastron

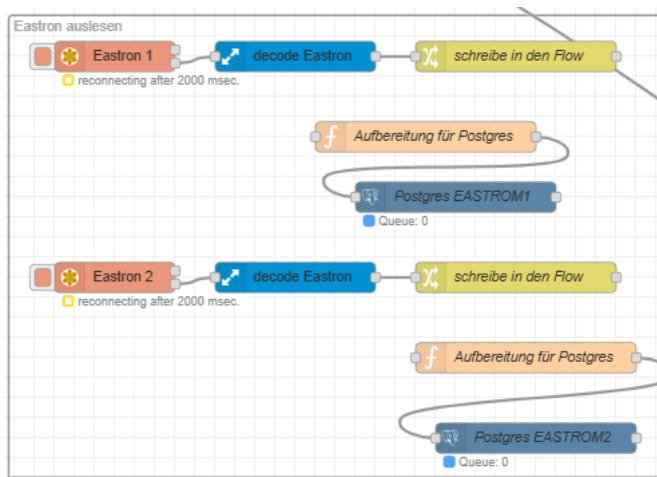


Abbildung 19: node-red Gruppe Eastron (Messgeräte)

In dem Eastron-Flow werden die Daten aus der Eastrons ausgelesen, dekodiert und angezeigt. Die Strom-Werte der Phasen werden außerdem direkt in den Flow-State geschrieben, um auch ohne eine direkte “Node-Red-Verbindung”, insbesondere auch asynchron, auf die Daten zugreifen zu können.

Das Format eines Datenpackets ist 8N1 mit einer Übertragungsrate von 9600. In der Ladesäule hat der linke Zähler die Adresse 34 und der rechte hat die Adresse 33. Die Werte, welche ausgelesen werden, sind dem „Parser“-Node entnehmbar und weitere Informationen finden sich im Datenblatt, welches im Dokumentationsordner zu finden ist.

An dieser Stelle ist eine Thematik wichtig. Werden beide Zähler mit einer Pollrate von 1s abgefragt, wie es bei den Controllern umgesetzt ist, so entstehen ununterbrochen Time-outs. Dabei handelt es sich mit hoher Wahrscheinlichkeit um Paketkollisionen, welche bei gleichzeitigem Transmit- und Recieve-Vorgang auftreten (unidirektionaler Bus). Da der Datenverkehr sich in diesem Fall in Grenzen hält, reicht es aus für einen Modbus-Read-Node einen „delay on request“ von einer halben Sekunde einzustellen. Außerdem ist die Pollrate auf 3s gesetzt, da dies für die Optimierung ausreichend ist. Bei mehr Busteilnehmern und höherem Datenverkehr ist es empfehlenswert den Modbus-Getter-Node zu verwenden, welcher die Vorgänge koordiniert (siehe Erläuterung in Node-Red).

3 Login-Daten und Zugriff auf den Ladesäulen-Pi

Der Zugriff auf den Raspberrypi in der Ladesäule ist nur im VPN möglich. Einen VPN-Zugang bekommt man bei Herrn Kerber. Der Login auf dem Pi ist nur über einen SSH-Key möglich, falls da ein neues installiert werden soll, bitte an Lukas Schulz wenden.

Das Node-Red-Admin-Interface ist im VPN unter [<http://10.21.6.6:1880>]<http://10.21.6.6:1880>] erreichbar.

Der Login dafür ist (auch für ssh):

Username: *admin*

Password: *lamaadmin1234*

“Admin-Interface”

Das “Admin-Interface” für Enduser ist unter <http://10.21.6.6:1880/ui/#!/1> erreichbar.

4 Datenbank Postgresql

Die Datenbank der Vorgängergruppe InfluxDB wurde mit der Datenbank Postgresql ersetzt. Der Vorteil der ehemals verwendeten Datenbank InfluxDB ist, dass es eine Time-Serie-Database ist, welche Messdaten mit einer höheren Schreibperformance besser arrangieren und für eine grafische Ausgabe über Grafana besser geeignet ist.

Postgresql ist eine relationale Datenbank, welche insgesamt universaler im Vergleich zu InfluxDB ist. Postgresql wurde gewählt, um relationale Daten zu verwalten und verarbeiten, also um mehrere Tabellen anlegen zu können, die miteinander verbunden sind. Diese Funktion ist für das Anmeldefunktion erforderlich.

Verwaltet werden können Postgresql-Datenbanken über die Anwendung pgAdmin 4 (graphisch ansprechender und einsteigerfreundlicher) oder rein über das Kommandoterminal.

Aktuell ist nur die benutzer-Tabelle auf der Datenbank des Raspberry in der Ladesäule. Die übrigen Tabellen wurden dort nicht angelegt. Dies hat den Grund, dass die Datenbank auf eine andere Stelle im Netzwerk ausgelagert werden soll und so die SD-Karte durch weniger Schreibzyklen langlebiger zu halten. Deswegen sind auch im node-red-flow die Postgres-nodes mit deren Aufbereitungs-function-nodes nicht angeschlossen. Sollte die „externe“ Datenbank später mit angebunden werden, dann sind die nodes unbedingt zu verbinden.

Im Folgenden werden die der SQL-Code zur Erstellung der genutzten aufgeführt, um die Struktur der Datenbank mit seinen Tabellen nachzuvollziehen und um die Datenbankstruktur bei Verlust oder Neuanlegung zu reproduzieren.

Benutzer-Tabelle:

```
CREATE TABLE benutzer(  
  karten_id VARCHAR(30) NOT NULL,  
  vorname VARCHAR(30) NOT NULL,  
  nachname VARCHAR(30) NOT NULL,  
  email VARCHAR(50) NULL,  
  auto VARCHAR(30) NULL,  
  erstelldatum DATE DEFAULT CURRENT_DATE,  
  status BOOLEAN DEFAULT TRUE,  
  benutzer_id SERIAL PRIMARY KEY  
);
```

Eastron-Tabelle (identisch für Eastron2, nur anderer Tabellename eastron2):

```
create table eastron1(  
  time timestamp default current_timestamp,  
  V_L1 float null,  
  V_L2 float null,  
  V_L3 float null,  
  V_L float null,  
  I_L1 float null,  
  I_L2 float null,
```

```

I_L3 float null,
I_L float null,
P_L1 float null,
P_L2 float null,
P_L3 float null,
P_L float null,
export_work float null,
time_id serial primary key
);

```

EVSE-Tabelle (identisch für EVSE2, nur anderer Tabellenname evse2):

```

create table evse1(
time timestamp default current_timestamp,
vehicle_state integer null,
actual_configured_amps integer null,
actual_amps integer null,
optimized_amps integer null,
time_id serial primary key
);

```

Janitza-Tabelle:

```

create table janitza(
time timestamp default current_timestamp,
V_L1 float null,
V_L2 float null,
V_L3 float null,
V_L float null,
I_L1 float null,
I_L2 float null,
I_L3 float null,
I_L float null,
P_L1 float null,
P_L2 float null,
P_L3 float null,
P_L float null,
time_id serial primary key
);

```

Ladevorgang-Tabelle (diese Tabelle wird aktuell nicht verwendet, könnte aber vtl. Nützlich sein, wenn man Ladevorgänge tracken will, z.B. für Bezahlssystem oder bezogene Gesamtleistung für einen Nutzer anzeigen lassen):

```

create table ladevorgang(
evse_id integer not null,
benutzer_id integer not null,
l_start timestamp default current_timestamp,
l_end timestamp default current_timestamp,
exported_work float null,
l_status boolean default false,
ladevorgang_id serial primary key
);

```

SQL-Syntax-Beispiele:

Manuell neuen Benutzer anlegen:

```
INSERT INTO benutzer (karten_id, vorname, nachname, email)
VALUES ('0987654321', 'Hans', 'Wurst', 'hans.wurst@hm.edu');
```

Manuel einen Benutzer löschen:

```
DELETE FROM benutzer
WHERE benutzer_id = '2';
```

Die Datenbank (allerdings nur die Tabelle benutzer) befindet sich nach Erstellung der Doku auf der SD-Karte des Raspberry, soll aber an einer Stelle im HM-Netzwerk ausgelagert werden. Wenn das geschieht, ist es wichtig, dass im Postgres-Node der Server geändert wird.

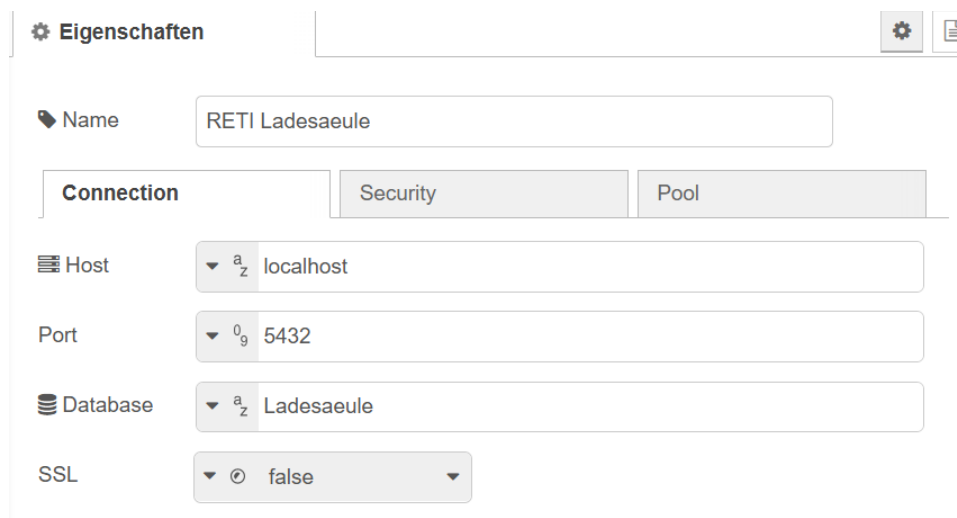
Die Einstellungen des aktuellen Servers auf der SD-Karte sind:

Name der Datenbank: lamaraspi

User: postgres

Passwort: lamaadmin1234

Weitere Einstellungen siehe folgende Abbildung:



The screenshot shows a configuration window titled 'Eigenschaften' (Properties) for a database connection. The 'Name' field is set to 'RETI Ladesaeule'. Below this, there are three tabs: 'Connection', 'Security', and 'Pool'. The 'Connection' tab is selected, showing the following settings: 'Host' is 'localhost', 'Port' is '5432', 'Database' is 'Ladesaeule', and 'SSL' is 'false'.

Abbildung 20: Eingestellte Connection-Eigenschaften des aktuellen Servers für die Datenbank auf der SD-Karte

Folgender Node-red-Abschnitt ist dafür zuständig, alle Daten der Messgeräte zu löschen, wenn sie älter als ein Jahr sind. Dies soll verhindern, dass die Datenbank einen zu hohen Speicherbedarf hat. Man hätte diese Einstellung auch in der Datenbank selbst vornehmen können, wurde allerdings aus persönlichen Präferenzen so umgesetzt.

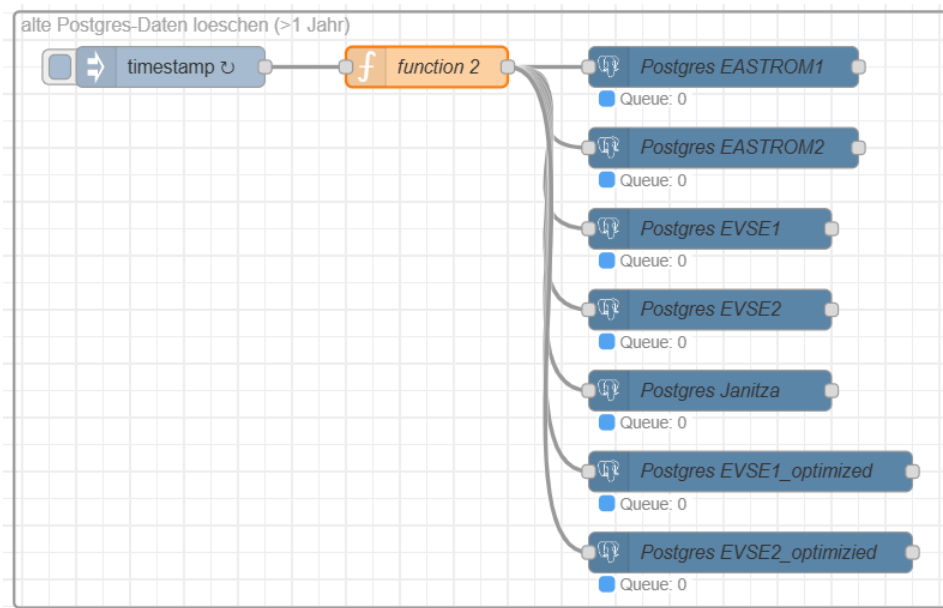


Abbildung 21: Node-red-Abschnitt Löschung von alten Messdaten, die älter als ein Jahr sind

In allen Gruppen in node-red, die ein Messgerät auslesen, ist einen function-node zur Aufbereitung mit nachfolgender Postgresql-node vorhanden. Diese speichern und loggen die Daten der Messgeräte in die Datenbank.